

MixBytes()

Lido SRv3 CMv2 Voting script Security Audit Report

JULY 09, 2026

Table of Contents

1. Introduction	2
1.1 Disclaimer	2
1.2 Executive Summary	2
1.3 Project Overview	4
1.4 Security Assessment Methodology	7
1.5 Risk Classification	10
1.6 Summary of Findings	11
2. Findings Report	13
2.1 Critical	13
2.2 High	13
2.3 Medium	13
M-1 EasyTrack factory changes are enacted independently of the Dual Governance upgrade	13
2.4 Low	15
L-1 _checkSRMFinalState no longer validates the new module's configuration	15
L-2 startUpgrade does not verify the protocol's pre-upgrade state	16
L-3 Consolidation migrator sourceModuleId is never cross-checked	17
L-4 Finalizer and setter parameters are not read back in finishUpgrade	18
L-5 finishUpgrade never asserts the Agent's Aragon APP_MANAGER_ROLE window was closed	19
L-6 _checkDSMMigration validates the guardian set in one direction only	20
L-7 finishUpgrade verifies only 4 of the new LidoLocator's service addresses	21
L-8 STAKING_MODULE_MANAGE_ROLE is verified by presence rather than exclusivity	22
L-9 _assertCMFinalState does not check Curated Module v2 proxy implementations or admins	23
L-10 Tight upgrade expiry lets a sub-RageQuit partial veto block the upgrade	24
3. About MixBytes	25

1. Introduction

1.1 Disclaimer

The audit makes no statements or warranties regarding the utility, safety, or security of the code, the suitability of the business model, investment advice, endorsement of the platform or its products, the regulatory regime for the business model, or any other claims about the fitness of the contracts for a particular purpose or their bug-free status.

1.2 Executive Summary

The audited code is the on-chain upgrade machinery for the Lido Staking Router v3 / Curated Module v2 release. `UpgradeConfig` ingests an `UpgradeParameters` struct, discovers the live protocol topology by reading getters on the new LidoLocator and the CSM/Curated modules, and exposes it as immutable configuration; `UpgradeVoteScript` builds the single Aragon vote that submits a Dual Governance proposal (the 69-item core/CSM/CM upgrade) and directly registers the EasyTrack factory changes; `UpgradeTemplate` brackets the proposal with `startUpgrade/finishUpgrade`, enforcing that they run in one transaction and asserting the protocol's final implementations, roles, and wiring. The main actors are the Aragon Voting, the Agent, and Dual Governance, with the contracts acting purely as a one-shot, self-disarming orchestrator rather than a long-lived protocol component.

The interim security review was conducted over 9 working days via manual code review and a proprietary AI-assisted analysis tool.

Our review covered both the core orchestration logic and its many on-chain integration points, since these contracts derive state from and issue privileged calls into a large set of external components (Aragon, Dual Governance, EasyTrack, the CircuitBreaker, and the upgraded protocol contracts themselves). Alongside this we worked through our standard checklist – access control, role exclusivity, reentrancy, integer truncation, and the consistency of configured parameters against their on-chain effects – paying particular attention to where the template's final-state assertions might be incomplete relative to the actions the vote performs. The protocol-specific vectors examined during the engagement are listed below.

The Aragon (Kernel, ACL, Voting, TokenManager), Dual Governance and EmergencyProtectedTimelock, and all of the upgraded protocol contracts (Lido, StakingRouter, the oracles, WithdrawalVault, CSM, the Curated Module, and the Consolidation/TopUp gateways) are invoked by the in-scope code but were themselves out of scope; we reviewed only how these contracts call into them and relied on their interfaces and published behavior, rather than auditing those dependencies line by line.

Below we set out our overall assessment, key assumptions, and main recommendations. Findings raised in this interim review are rated **Low** to **Medium** severity under our Impact x Likelihood classification: they are predominantly missing or weakened verification checks whose realization depends on a misconfiguration introduced by the trusted upgrade author or on a governance-path failure, rather than directly exploitable conditions. The single Medium finding concerns the

EasyTrack factory changes being enacted independently of the Dual Governance upgrade, which can leave the factory registry diverged from the live protocol until reconciled by a follow-up governance action. The contracts' correctness ultimately relies on careful off-chain review of the upgrade parameters in combination with the on-chain `finishUpgrade` assertions.

- **Self-disarming, one-shot design is sound.** The template's transient-storage same-transaction guard and expiry timestamp are a clean way to prevent a half-applied upgrade, access is correctly restricted to the Agent, and the canonical `UpgradeVoteScript` bundles `startUpgrade` and `finishUpgrade` into a single Dual Governance proposal that executes atomically. The one-shot, non-recoverable nature of the contract is noted as a minor observation below.
- **Final-state assertions are thorough but not exhaustive.** `finishUpgrade` re-derives and checks a large surface of implementations, proxy admins, role holders, and cross-references. However, several values the vote actually sets are never read back (e.g. the Triggerable Withdrawals Gateway limits and `maxTopUpPerBlockGwei`), the staking-module final-state check was reduced to verifying only the new module's address, the transient Aragon `APP_MANAGER_ROLE` window opened to repoint the Lido base is not asserted closed (unlike every other transient role window), and one powerful role is verified by presence rather than exclusivity. The pre-upgrade baseline checks in `startUpgrade` were also removed at this commit, placing the full verification burden on the final-state assertions.
- **Validation is anchored on off-chain trust in parameters.** Because `UpgradeConfig` derives nearly every critical address from the supplied `newLocatorImpl`, the integrity of the whole upgrade rests on careful off-chain review of the constructor parameters; the contracts perform no independent cross-check that the discovered addresses match the live pre-upgrade protocol.
- **EasyTrack and Dual Governance halves of the vote are not coupled.** Factory registrations take effect on vote enactment while the upgrade itself only enters the DG timelock, so the two can diverge if the DG proposal is vetoed or stalls.
- **Deploy-to-enactment timing is fragile.** Some values are fixed at deployment but only consumed at enactment, after the Aragon vote and the Dual Governance timelock. In particular, if a bond curve is added to Accounting between deployment and enactment, the new curve's id shifts (ids are assigned sequentially), and because the expected id is baked in at deployment the upgrade reverts at enactment. The failure mode is liveness-only (a caught revert, not silent corruption), but it requires minimizing and guarding the deploy-to-enactment window.
- **Minor observation (below Low severity).** The template is a one-shot contract with no reset path, so an Agent action or malformed proposal that called `startUpgrade` without `finishUpgrade` in the same transaction would permanently disarm it and force a redeploy (the canonical `UpgradeVoteScript` prevents this by bundling both calls into one atomically-executed proposal). This is governance/Agent-only and recoverable by redeploy, and is recorded here rather than as a numbered finding.
- **Recommendations.** Restore the full module-configuration check and the `startUpgrade` pre-condition assertions, read back every parameter the vote sets, assert role exclusivity (not just presence) for high-privilege roles on mainnet, add an explicit `newEjector != oldEjector` guard, and consider committing to the expected vote-item set rather than only its count.

1.3 Project Overview

Summary

Title	Description
Client	Lido SRv3
Category	Liquid Staking
Project	CMv2 Voting script
Type	Solidity
Platform	EVM
Timeline	17.06.2026 - 09.07.2026

Scope of Audit

File	Link
contracts/upgrade/UpgradeConfig.sol	contracts/upgrade/UpgradeConfig.sol
contracts/upgrade/UpgradeTemplate.sol	contracts/upgrade/UpgradeTemplate.sol
contracts/upgrade/UpgradeVoteScript.sol	contracts/upgrade/UpgradeVoteScript.sol
src/utils/OneShotCurveSetup.sol	src/utils/OneShotCurveSetup.sol

Versions Log

Date	Commit Hash	Note
17.06.2026	a744bd1eb7f661604a2f031d29e10bb69027b26c	Initial Commit (Core)
17.06.2026	85f20f90324b3262749eb74d450ee436fef80fe9	Initial Commit (Staking Modules)
29.06.2026	3436e09c8dc8cba90accdec7d00b8b7441ed438f	Commit with Updates (Core)

Date	Commit Hash	Note
06.07.2026	9d6a14145265dea095faf4f550c8a84fa445f536	Commit for Re-audit (Core)
09.07.2026	4c889ca95f66a8d57fb8d1eb2b83d251e51f8718	Commit with Updates (Core)

Mainnet Deployments

File	Address
UpgradeConfig.sol	0xf94e7A95C559728AcA33bFa7136Ea5De40B0338E
UpgradeTemplate.sol	0xD92b6303Ba39297Cb69a3a17A88b47586A6af14C
UpgradeVoteScript.sol	0xE6530830A2cf90773cB232748b2c674c27b6E0CA
OneShotCurveSetup.sol	0x711985E069f4d702e0457C0dACAd3D3894Ce4E3

The deployment was reviewed against [4c889ca95f66a8d57fb8d1eb2b83d251e51f8718](#), the commit that introduced these contracts into the [deployed-mainnet.json](#) deployment artifact. That commit has the audited re-audit commit [9d6a14145265dea095faf4f550c8a84fa445f536](#) as a direct ancestor, and its [contracts/](#) tree is byte-for-byte identical to the audited scope (no diff under [contracts/](#) between the two), so the deployed source is the audited source. All three contracts are verified on Etherscan with matching compiler settings, and none is a proxy, consistent with their one-shot orchestrator design.

The audited contracts and their upgrade-package helpers and shared protocol interfaces/libraries diff clean against the deploy commit, and every OpenZeppelin dependency file matches the exact pinned upstream versions. Dependencies are pinned with no semver ranges:

[@openzeppelin/contracts-v4.4](#) resolves to [@openzeppelin/contracts@4.4.1](#) and [@openzeppelin/contracts-v5.2](#) to [@openzeppelin/contracts@5.2.0](#) (both compared byte-for-byte against the published npm packages), and the Aragon dependencies are pinned to exact versions as well.

The on-chain wiring and configuration match the deployment artifact. [UpgradeVoteScript](#) reports [TEMPLATE](#) and [CONFIG](#) pointing at the deployed [UpgradeTemplate](#) and [UpgradeConfig](#); [UpgradeTemplate.CONFIG](#) points at the same [UpgradeConfig](#); and [UpgradeConfig.AGENT](#) is the Lido DAO Aragon Agent ([0x3e40D73EB977Dc6a537af587D48316feE66E9C8c](#)), which is also the [ADMIN](#) of the live CircuitBreaker ([0x6019CB557978296BA3C08a7B73225C0975DFB2F7](#)) referenced by the config, so the sole privileged actor across the machinery is the governance-controlled Agent. Neither [UpgradeConfig](#) nor [UpgradeVoteScript](#) exposes any privileged role (no [owner](#), [AccessControl](#), [admin](#), or [pauser](#)), and [UpgradeTemplate](#) has no owner or reset path: its only privileged entrypoints, [startUpgrade/finishUpgrade](#), are Agent-gated and further guarded by same-transaction and expiry checks. The config addresses ([LOCATOR](#), [VOTING](#), [DUAL_GOVERNANCE](#), [CIRCUIT_BREAKER](#), [CIRCUIT_BREAKER_COMMITTEE](#), [BURNER](#)) and parameters read back on-chain match the constructor arguments in [deployed-mainnet.json](#), including the Triggerable Withdrawals Gateway limits, the Accounting Oracle/VEBO consensus versions and limits, and the Curated Module v2 fees, share limits, [moduleName](#) ([curated-onchain-v2](#)), and [hashConsensusInitialEpoch](#). [EXPIRE_SINCE_INCLUSIVE](#)

reads back as [1786147199](#) (August 7, 2026, 23:59:59 UTC), the widened value the client committed to when resolving Low #10.

At the time of review `isUpgradeFinished` is `false`: the machinery has not been enacted, so the on-chain state is the constructor baseline only. The final protocol state asserted by `finishUpgrade` (implementations, proxy admins, role holders, and the pauser-to-pausable bindings) is established only when the Lido DAO vote executes.

`OneShotCurveSetup.sol` is the supplementary helper from the [lidofinance/staking-modules](#) repository (the `identifiedDVTCurveSetup` at [0x711985E069f4d702e0457C0dACAd3D3894Ce4E3](#), which seeds the bond curve with id 3 into the CSM Accounting). Its deployed source is byte-for-byte identical to the audited contract at commit [85f20f90324b3262749eb74d450ee436fef80fe9](#): the `OneShotCurveSetup.sol` blob and its `AssetRecovererLib.sol` library match the audit commit exactly. The full verified source set was compiled from a later commit ([31aa6e14c4701e4443992e13a4f19f6030af2a88](#)), against which all bundled files diff clean; the only deviations from the audit commit are NatSpec and interface-doc changes in supporting `interfaces/*` files (comment wording, a renamed unused parameter, and two removed unused error selectors), none of which the deployed helper references, so they have no effect on its bytecode. The helper's dependencies are pinned: `@openzeppelin/contracts` is fixed to `5.4.0` in `package.json`, and every OpenZeppelin file in the verified source matches that published package byte-for-byte. It is verified on Etherscan (solc `0.8.33`, non-proxy), and its `ACCOUNTING` and `REGISTRY` immutables read back on-chain as [0x4d72BFF1BeaC69925F8Bd12526a39BAAb069e5Da](#) and [0x9D28ad303C90DF524BA960d7a2DAC56DcC31e428](#), which are the `accounting` and `parametersRegistry` entries of the config's `getCSMUpgradeConfig()` (the CSM Accounting and CSM ParametersRegistry `OssifiableProxy` contracts), confirming the setup targets the CSM module it seeds bond curve id 3 into. It carries no owner or standing privileged role: its `execute()` renounces the temporary `MANAGE_BOND_CURVES_ROLE` and `MANAGE_CURVE_PARAMETERS_ROLE` it is granted, once used.

1.4 Security Assessment Methodology

Stage 1: Interim Audit

Project Architecture Review

OBJECTIVE: UNDERSTAND THE OVERALL STRUCTURE OF THE PROTOCOL AND IDENTIFY POTENTIAL SECURITY RISKS, INCLUDING PRIMITIVES AND ABSTRACTIONS IMPLEMENTED BY THE SYSTEM, HOW THEY INTERACT ARCHITECTURALLY, AND WHERE DESIGN OR INTEGRATION WEAKNESSES COULD BE EXPLOITED.

Activities include:

- understanding overall system design and contract interactions
- mapping responsibilities of each component and protocol workflows
- conducting a thorough line-by-line review of contracts and modules
- tracing execution paths and mapping state transitions
- identifying trust boundaries and external integrations
- forming an independent architectural understanding directly from code before validating documentation
- running the project test suite to observe implementation behavior and encoded invariants
- reviewing documentation, specifications, READMEs, and deployment guides and reconciling them with code

Adversarial Code Review

OBJECTIVE: IDENTIFY POTENTIAL VULNERABILITIES SPECIFIC TO THE PROTOCOL ARCHITECTURE, BUSINESS LOGIC, AND ECONOMIC MECHANISMS.

Approach includes:

- analyzing the codebase from an attacker's perspective, focusing on what can fail rather than only confirming intended behavior
- searching for overlooked edge cases, invalid assumptions, unexpected call sequences, and unsafe state transitions
- attempting to violate protocol invariants and identifying scenarios where system guarantees break
- analyzing economic attack surfaces including liquidation logic, oracle dependencies, vault accounting, and cross-contract interactions
- writing targeted tests, modelling adversarial scenarios, applying mutation testing and fuzzing, and developing proof-of-concept exploits
- conducting independent exploit path discovery by each auditor to reduce anchoring bias
- reviewing high-risk code collaboratively in pair-auditing sessions led by the Team Lead

Systematic Vulnerability Analysis

OBJECTIVE: APPLY STRUCTURED ANALYSIS AND KNOWN ATTACK PATTERNS TO ENSURE COMPREHENSIVE COVERAGE ACROSS THE CODEBASE.

Activities include:

- reviewing code against an internally maintained vulnerability checklist derived from past exploits and research
- analyzing common DeFi attack classes such as reentrancy, accounting manipulation, oracle manipulation, privilege escalation, and state desynchronization
- using proprietary AI tooling to surface candidate issues across broader attack vectors
- manually validating and triaging AI-generated candidates

Consolidation of Auditors' Reports

OBJECTIVE: MERGE INTERIM INPUTS FROM ALL AUDITORS INTO A COHERENT REPORT WITH CONSISTENT SEVERITY LEVELS AND REPRODUCIBLE FINDINGS.

Activities include:

- cross-checking findings across auditors
- consolidating and deduplicating issues
- resolving discrepancies in severity assessments
- using proprietary AI tooling to assist with report drafting and consistency checks
- manually reviewing and finalizing the report narrative
- preparing the interim audit report for client review

Stage 2: Re-audit & Mainnet Deployment Verification

Re-audit

OBJECTIVE: VERIFY THAT CLIENT REMEDIATIONS ARE CORRECTLY IMPLEMENTED AND THAT THE UPDATED CODE DOES NOT INTRODUCE NEW VULNERABILITIES.

Activities include:

- confirming each remediation matches the original recommendation
- documenting rationale for any unresolved findings
- verifying modified logic and integration points remain secure
- executing the project's tests after remediation, including targeted coverage of fixed areas
- using proprietary AI tooling on the updated codebase to surface potential new issues

Mainnet Deployment Verification

OBJECTIVE: PERFORM FINAL VERIFICATION OF DEPLOYED CONTRACTS AND CONFIGURATION BEFORE ISSUING THE PUBLIC AUDIT REPORT.

Activities include:

- verifying deployed contract bytecode against audited source code across networks
- confirming deployed bytecode matches the build produced from the audited commit using identical compiler settings
- reviewing constructor and initializer arguments used in deployment
- verifying proxy deployment order, initialization flow, and admin configuration
- ensuring implementations are not left uninitialized or exposed to initialization front-running
- publishing the final report after alignment between client and audit team

1.5 Risk Classification

Severity Level Matrix

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact

- **High** – Theft exceeding 0.5% of the protocol's TVL, partial or complete blocking of funds on the contract without the possibility of withdrawal (>0.5%), or loss of user funds exceeding 1% for users interacting with the protocol.
- **Medium** – Contract lock that can only be resolved through a contract upgrade, one-time theft of rewards or an amount up to 0.5% of the protocol's TVL, or funds lock where withdrawal is still possible by an administrator.
- **Low** – One-time contract lock that can be resolved by an administrator without requiring a contract upgrade.

Likelihood

- **High** – An event with an estimated 50-60% probability of occurring within a year, which can be triggered by any actor (e.g., due to a market condition that the actor cannot influence).
- **Medium** – An unlikely event (10-20% probability of occurring) that can be triggered by a trusted actor.
- **Low** – A highly unlikely event that can only be triggered by the contract owner.

Action Required

- **Critical** – Must be fixed as soon as possible.
- **High** – Strongly advised to be fixed in order to minimize potential risks.
- **Medium** – Recommended to be fixed to improve security and stability.
- **Low** – Recommended to be fixed to improve overall robustness and efficiency.

Finding Status

- **Fixed** – The recommended fixes have been implemented in the project code and the issue no longer impacts the security of the protocol.
- **Partially Fixed** – The recommended fixes have been partially implemented, reducing the impact of the finding, but the issue has not been fully resolved.
- **Acknowledged** – The recommended fixes have not been implemented, and the finding remains unresolved or has been accepted by the project team without code changes.

1.6 Summary of Findings

Findings Count

Severity	Count
Critical	0
High	0
Medium	1
Low	10

Findings Statuses

ID	Finding	Severity	Status
M-1	EasyTrack factory changes are enacted independently of the Dual Governance upgrade	Medium	Acknowledged
L-1	<code>_checkSRMFinalState</code> no longer validates the new module's configuration	Low	Acknowledged
L-2	<code>startUpgrade</code> does not verify the protocol's pre-upgrade state	Low	Acknowledged
L-3	Consolidation migrator <code>sourceModuleId</code> is never cross-checked	Low	Acknowledged
L-4	Finalizer and setter parameters are not read back in <code>finishUpgrade</code>	Low	Acknowledged
L-5	<code>finishUpgrade</code> never asserts the Agent's Aragon <code>APP_MANAGER_ROLE</code> window was closed	Low	Fixed
L-6	<code>_checkDSMMigration</code> validates the guardian set in one direction only	Low	Fixed
L-7	<code>finishUpgrade</code> verifies only 4 of the new LidoLocator's service addresses	Low	Acknowledged
L-8	<code>STAKING_MODULE_MANAGE_ROLE</code> is verified by presence rather than exclusivity	Low	Fixed

L-9	<code>_assertCMFinalState</code> does not check Curated Module v2 proxy implementations or admins	Low	Fixed
L-10	Tight upgrade expiry lets a sub-RageQuit partial veto block the upgrade	Low	Acknowledged

2. Findings Report

2.1 Critical

Not Found

2.2 High

Not Found

2.3 Medium

M-1	EasyTrack factory changes are enacted independently of the Dual Governance upgrade		
Severity	Medium	Status	Acknowledged

Description

The Aragon vote built by `getEVMScript` is a `submitProposal` call followed by the EasyTrack voting items (`appended directly`). These execute on two different timelines: `submitProposal` only queues the upgrade into the Dual Governance timelock, while the EasyTrack `addEVMScriptFactory/removeEVMScriptFactory` items run immediately when the vote is enacted.

The factory changes therefore become live the moment the vote passes, even if the Dual Governance proposal is later vetoed, never scheduled, or never executed. Nothing reverts them if the DG half does not complete, so the new motion factories can end up registered against the pre-upgrade protocol.

In practice the registered factories are inert until the upgrade executes: the roles they ultimately rely on (e.g. `STAKING_MODULE_SHARE_MANAGE_ROLE` on the StakingRouter) are granted inside the DG proposal body, not the voting items, and the target contracts/methods only come into existence through the DG-gated upgrade. A motion routed through such a factory before DG execution would revert. The residual effect is a temporary window of inconsistency in the EasyTrack factory registry (including the early removal of two old factories), not an exploitable action against the pre-upgrade protocol.

This issue was assigned Medium severity: the EasyTrack factory registry can be left in a state that diverges from the live protocol if the Dual Governance half is vetoed or stalls, requiring a follow-up governance action to reconcile rather than resolving on its own (Medium impact), and realization requires the Dual Governance half to be vetoed or to stall while the Aragon vote passes (Low likelihood).

Recommendation

We recommend documenting and accepting the decoupling explicitly, or gating the EasyTrack factory registrations so they cannot take effect unless the corresponding upgrade has executed (e.g. registering factories as part of the DG proposal body rather than the Aragon voting items).

Client's Commentary:

Status: Acknowledged.

The EasyTracks we plan to deploy do not make sense without the contract upgrade. We also do not think it is worth complicating the implementation of the voting script solely to activate the EasyTracks only after the upgrade has passed through Dual Governance.

2.4 Low

L-1	<code>_checkSRMFinalState</code> no longer validates the new module's configuration		
Severity	Low	Status	Acknowledged

Description

In `_checkSRMFinalState`, the check on the newly added module was reduced to a single field, `moduleAddress`. The prior version also asserted `moduleFee`, `treasuryFee`, `stakeShareLimit`, `priorityExitShareThreshold`, `status == Active`, and `withdrawalCredentialsType == 0x02`; none of these are re-checked anymore.

StakingRouter/SRLib still reject out-of-bounds values (fees over `TOTAL_BASIS_POINTS`, bad share ordering, zero deposit distance), so an invalid configuration reverts. But a valid-yet-wrong configuration for Curated Module v2, e.g. the wrong fee split or share limit, passes the upgrade and surfaces only in operation. The `setName` on the Identified Community Stakers gate (`UpgradeVoteScript.sol:553`) is likewise not read back.

This issue was assigned Low severity: a wrong-but-valid fee/share configuration mis-sets module economics and is correctable by a follow-up governance action rather than causing direct theft or fund lock (Low impact), and it requires a parameter error that still passes SRLib bounds, set by the trusted upgrade author (Low likelihood).

Recommendation

We recommend restoring the full `ModuleStateConfig` comparison for the newly added module in `_checkSRMFinalState`, asserting `moduleFee`, `treasuryFee`, `stakeShareLimit`, `priorityExitShareThreshold`, `status`, and `withdrawalCredentialsType` against the configured `CuratedModuleConfig` values, and asserting the gate name set by `setName`.

Client's Commentary:

Status: Acknowledged.

The on-chain validation performed by the upgrade template is primarily focused on verifying security-critical invariants rather than validating every configuration parameter.

In addition, we perform extensive off-chain validation before deployment by simulating the entire vote and verifying that all expected parameter values are set correctly. After the vote is executed, we perform another off-chain verification of all configured parameters to ensure the protocol has been upgraded as expected.

L-2	startUpgrade does not verify the protocol's pre-upgrade state		
Severity	Low	Status	Acknowledged

Description

startUpgrade performs only the caller, expiry, and not-already-started guards, then snapshots a few protocol values (buffered ether, beacon stats, withdrawal credentials, module count) for later comparison. It does not verify that any proxy to be upgraded is currently running its expected old implementation, nor that the Aragon kernel points at the expected Lido base. The upgrade therefore begins mutating implementations without confirming the protocol is in the assumed starting state. If the live implementations differ from what the supplied parameters assume, the upgrade proceeds from an unverified baseline and finishUpgrade alone must catch every resulting divergence.

This issue was assigned Low severity: it represents a loss of defense-in-depth, but finishUpgrade still asserts the final state extensively so most baseline drift is still caught at the end (Low/Medium impact), and realization requires owner-only execution against a protocol whose live implementations have diverged from the assumed parameters (Low likelihood).

Recommendation

We recommend adding pre-condition checks in startUpgrade (the current implementation per proxy and the Aragon kernel Lido base) so the upgrade refuses to start unless the protocol matches the assumed baseline, providing defense-in-depth alongside the finishUpgrade assertions.

Client's Commentary:

Status: Acknowledged.

Due to our deployment process, this scenario is not expected to occur in practice. We do not perform multiple protocol upgrades concurrently, and no other governance votes are expected to modify the relevant contracts, implementations, or addresses between the start of voting and upgrade execution.

Therefore, the protocol state is expected to remain unchanged during the whole voting and activation period.

L-3	Consolidation migrator <code>sourceModuleId</code> is never cross-checked		
Severity	Low	Status	Acknowledged

Description

`_checkSRMFinalState` asserts the migrator's `targetModuleId()` but never checks `sourceModuleId()`. Both are non-zero immutables (constructor-guarded), so a wrong-but-non-zero source passes the upgrade and only fails later when `_getModule(SOURCE_MODULE_ID)` runs. Given the migrator's privileged `ALLOW_PAIR/PUBLISH` wiring, the source identity is worth asserting here too. This issue was assigned Low severity: a wrong-but-non-zero source module causes misdirected consolidations that fail at runtime rather than losing funds (Low impact), and realization depends on a deploy-time migrator misconfiguration (Low likelihood).

Recommendation

We recommend adding an assertion that `IConsolidationMigrator(consolidationMigrator).sourceModuleId()` equals the configured expected source module id.

Client's Commentary:

Status: Acknowledged.

We do not consider an additional on-chain assertion of `sourceModuleId` necessary. The source module already exists in the protocol, and its module ID is fixed. This value is verified during the deployment process and cannot change throughout the governance process.

In contrast, `targetModuleId` is assigned only when the new staking module is added to the Staking Router. Therefore, we explicitly verify `targetModuleId` after the upgrade to ensure that the migrator is configured to reference the newly added module.

L-4	Finalizer and setter parameters are not read back in <code>finishUpgrade</code>		
Severity	Low	Status	Acknowledged

Description

The vote feeds several config values into finalizer and setter calls, but `finishUpgrade` asserts only the associated version bump or role, never the applied value:

- `setExitRequestLimit` on the Triggerable Withdrawals Gateway: `twMaxExitRequestsLimit`, `twExitsPerFrame`, `twFrameDurationInSec` are not read back (only `TW_EXIT_LIMIT_MANAGER_ROLE` is asserted).
- `finalizeUpgrade_v3` on ValidatorsExitBusOracle: `veboMaxValidatorsPerReport`, `veboMaxExitBalanceEth`, `veboBalancePerFrameEth`, `veboFrameDurationInSec` are not asserted (only the consensus version is checked).
- `finalizeUpgrade_v4` on StakingRouter: `maxTopUpPerBlockGwei` is not asserted (only the contract version).
- `setEjector` on ValidatorStrikes: there is no `strikes.ejector() == newEjector` check.

A wrong-but-valid value for any of these is applied and passes the upgrade silently, surfacing only in operation.

This issue was assigned Low severity: a wrong limit or finalizer parameter is an operational misconfiguration tunable later by governance rather than a fund loss (Low impact), and realization requires a parameter error by the trusted upgrade author (Low likelihood).

Recommendation

We recommend reading back each applied value via its getter and comparing it to the `UpgradeConfig` value, mirroring the existing `hashConsensusInitialEpoch` pattern. At minimum, assert the TW gateway exit-request limits, the VEBO and StakingRouter finalize parameters, and `strikes.ejector()`.

Client's Commentary:

Status: Acknowledged.

The on-chain validation performed by the upgrade template is primarily focused on verifying security-critical invariants rather than validating every configuration parameter. We rely on the underlying protocol contracts, which have already undergone comprehensive security audits, to correctly apply and validate these configuration values.

In addition, we perform extensive off-chain validation before deployment by simulating the entire governance vote and verifying that all expected parameter values are set correctly. After the vote is executed, we perform another off-chain verification of all configured parameters to ensure the protocol has been upgraded as expected.

L-5	finishUpgrade never asserts the Agent's Aragon APP_MANAGER_ROLE window was closed		
Severity	Low	Status	Fixed in 9d6a1414

Description

To replace the Lido base in the Aragon Kernel, the vote temporarily grants APP_MANAGER_ROLE to the Agent, calls setApp, then revokes the role. APP_MANAGER_ROLE grants kernel-wide setApp power, i.e. the ability to replace any Aragon app base, including Lido/stETH. finishUpgrade closes every other temporary role window it opens with a negative assertion (e.g. MANAGE_WITHDRAWAL_CREDENTIALS_ROLE via _assertZeroOZRoleHolders). The APP_MANAGER_ROLE window is the exception: _assertCoreFinalState never asserts hasPermission(agent, kernel, APP_MANAGER_ROLE) == false.

The audited script revokes the role correctly, so there is no exploit path today. The gap is that the final-state checks would not catch a future or mis-built script in which the revoke item is dropped or mis-ordered, leaving the Agent with a standing kernel-wide setApp capability. This issue was assigned Low severity: it is a completeness gap in the final-state checks, not an exploitable condition, since the audited script closes the window correctly (Low likelihood); the potential impact, if a mis-built script left the role granted, is kernel-wide setApp power over any Aragon app base (High impact).

Recommendation

We recommend asserting in _assertCoreFinalState that the Agent no longer holds the role, e.g. `if (IAragonACL(c.acl).hasPermission(agent, c.kernel, APP_MANAGER_ROLE)) revert ...;` mirroring the pattern already used to close the other temporary role windows.

Client's Commentary:

Status: Fixed in commit 9d6a14145265dea095faf4f550c8a84fa445f536

L-6	<code>_checkDSMMigration</code> validates the guardian set in one direction only		
Severity	Low	Status	Fixed in 9d6a1414

Description

`_checkDSMMigration` verifies the DepositSecurityModule migration with three checks: the new DSM owner equals the Agent, the new quorum equals the old quorum, and every guardian in the new DSM was also a guardian in the old DSM.

The loop only proves `new ⊆ old`. It never checks the reverse direction or compares set sizes, so a strict subset of the old guardians at an unchanged quorum passes validation. For example, old `[A, B, C, D, E, F]` with quorum 4 and new `[A, B, C, D]` with quorum 4 is accepted even though two guardians were silently dropped.

This issue was assigned Low severity: a smaller-but-valid guardian set weakens deposit-security redundancy at an unchanged quorum without locking or losing funds (Low impact), and realization requires a new DSM deployed with a reduced guardian set (Low likelihood).

Recommendation

We recommend adding a length-equality assertion (`dsm.getGuardians().length == oldDsm.getGuardians().length`), which together with the existing subset loop enforces set equality so the guardian set cannot silently shrink or grow.

Client's Commentary:

Status: Fixed in commit [9d6a14145265dea095faf4f550c8a84fa445f536](#)

L-7	<code>finishUpgrade</code> verifies only 4 of the new LidoLocator's service addresses		
Severity	Low	Status	Acknowledged

Description

The vote upgrades the LidoLocator proxy to a new implementation bundling 24 immutable service addresses. `finishUpgrade` independently asserts only four via `_assertLocatorAddress`: `depositSecurityModule`, `consolidationGateway`, `topUpGateway`, and `oracleReportSanityChecker`. A second group (`lido`, `accounting`, `accountingOracle`, `stakingRouter`, `validatorsExitBusOracle`, `withdrawalVault`, `triggerableWithdrawalsGateway`) is only circularly covered: their addresses are read from the new locator in `UpgradeConfig`, and the template checks the target contract's implementation/admin/version, confirming "the proxy runs the expected implementation" rather than "the locator points to the right address." The remaining service addresses (`burner`, `treasury`, `elRewardsVault`, `withdrawalQueue`, and others) are not verified at all. `burner` is the most consequential: the vote grants `REQUEST_BURN_MY_STETH_ROLE` on `g.burner` and `finishUpgrade` asserts that role on the same `g.burner`, both reading `burner` from the new locator. A wrong `burner` in the new locator is therefore self-consistent and passes the upgrade, while the protocol's real burner never receives the roles. This issue was assigned Low severity: a wrong core service would break the protocol immediately and the locator is reviewed off-chain, but the on-chain template does not verify the address book it relies on, including the `burner` it grants roles on (Low impact); realization requires a deploy-time error in the new locator (Low likelihood).

Recommendation

We recommend asserting every LidoLocator service address against an independently-sourced expected value, or at minimum asserting that each unchanged field equals the previous locator's value and that `burner` equals the address the role grants target.

Client's Commentary:

Status: Acknowledged.

The on-chain validation performed by the upgrade template is primarily focused on verifying security-critical invariants rather than validating every configuration parameter.

During this upgrade, only four LidoLocator addresses are expected to change: `depositSecurityModule`, `oracleReportSanityChecker`, `consolidationGateway`, and `topUpGateway`. Therefore, only these addresses are explicitly verified on-chain. All other LidoLocator service addresses remain unchanged as part of this upgrade.

In addition, we perform extensive off-chain validation before deployment by simulating the entire governance vote and verifying that all expected parameter values are set correctly. After the vote is executed, we perform another off-chain verification of all configured parameters to ensure the protocol has been upgraded as expected.

L-8	STAKING_MODULE_MANAGE_ROLE is verified by presence rather than exclusivity		
Severity	Low	Status	Fixed in 9d6a1414

Description

In `_assertCoreFinalState`, every `StakingRouter` role is verified to an exact holder set via `_assertSingleOZRoleHolder` (or `_assertZeroOZRoleHolders`), except `STAKING_MODULE_MANAGE_ROLE`, which uses the weaker `_assertHasOZRole(sr, STAKING_MODULE_MANAGE_ROLE, agent)`, preceded by the comment `/// @dev _assertSingleOZRoleHolder not works on hoodi!`.

`STAKING_MODULE_MANAGE_ROLE` can add staking modules and update their parameters (share limits, fees, deposit settings). The presence check only confirms the Agent holds the role and does not bound the holder set, so an additional (stale, unexpected, or malicious) holder passes the upgrade undetected, even though every sibling role on the same contract is verified exactly. The relaxation is a testnet artifact (on Hoodi the role has two holders) carried into the mainnet code path.

This issue was assigned Low severity: on mainnet the role is expected to be held only by the Agent and holders are governance-controlled (Low impact), but the template silently accepts an unexpected holder of the most powerful `StakingRouter` role; realization requires an unexpected holder being present at upgrade time (Low likelihood).

Recommendation

We recommend asserting the full expected holder set for `STAKING_MODULE_MANAGE_ROLE` on mainnet (e.g. `_assertSingleOZRoleHolder`), rather than dropping the count check, so this role is verified as strictly as its siblings.

Client's Commentary:

Status: Fixed in commit [9d6a14145265dea095faf4f550c8a84fa445f536](#)

L-9	<code>_assertCMFinalState</code> does not check Curated Module v2 proxy implementations or admins		
Severity	Low	Status	Fixed in 9d6a1414

Description

Unlike `_assertCSMFinalState`, which verifies both the implementation (`_assertProxyImplementation`) and the proxy admin (`_assertProxyAdmin`) for every CSM proxy, `_assertCMFinalState` performs neither check for the Curated Module v2 contracts (`module`, `parametersRegistry`, `accounting`, `feeDistributor`, `feeOracle`, `strikes`); it validates only initialized versions, roles, and pausers. These contracts sit behind the same `OssifiableProxy` (exposing `proxy__getAdmin()` / `proxy__getImplementation()`), so a misconfigured proxy admin or implementation, e.g. the admin not handed off to the Agent during deployment, passes the upgrade silently rather than being caught by `finishUpgrade`.

This issue was assigned Low severity: a wrong CMv2 proxy admin or implementation is a deploy-time misconfiguration correctable by governance rather than a direct fund loss (Low impact), and realization requires a deployment error in the CMv2 proxies (Low likelihood).

Recommendation

We recommend adding `_assertProxyImplementation` and `_assertProxyAdmin(..., agent)` assertions for all CMv2 proxies in `_assertCMFinalState`, mirroring the checks `_assertCSMFinalState` already performs for the CSM proxies.

Client's Commentary:

Client:

Status: Fixed in commit [d3ab78fe97cd2ea16f0ad738efafc3f686221b99](#)

MixBytes: The proxy-admin assertion was added on parity with `_assertCSMFinalState`; the implementation assertion was intentionally omitted, which we consider well-reasoned since CM v2 proxy implementations are set and verified at deployment (unlike CSM, which is upgraded by the vote)

L-10	Tight upgrade expiry lets a sub-RageQuit partial veto block the upgrade		
Severity	Low	Status	Acknowledged

Description

`UpgradeTemplate` stores an immutable `EXPIRE_SINCE_INCLUSIVE` (currently `1785542399`, July 31, 2026, set in `scripts/upgrade/upgrade-params-mainnet.toml`), and `startUpgrade` reverts with `Expired` once `block.timestamp >= EXPIRE_SINCE_INCLUSIVE`. The Dual Governance proposal carrying the upgrade executes only after the dynamic timelock, which stETH holders can extend via `VetoSignalling` up to `veto_signalling_max_duration` (45 days on mainnet), scaling linearly between the first seal (1% rage-quit support) and the second seal (10%). Because the expiry is evaluated at execution time, delaying execution past `EXPIRE_SINCE_INCLUSIVE` makes the proposal permanently unexecutable, since the template is single-use and the deadline is immutable. This subverts the two-tier design of Dual Governance: `VetoSignalling` ($\geq 1\%$) is intended only to delay a proposal, whereas only `RageQuit` ($\geq 10\%$) should permanently block it. When the margin between the proposal's execution window and the expiry is smaller than the maximum achievable delay, a partial veto below the RageQuit threshold (as low as 1-5%, depending on the margin) is enough to push execution past the deadline, without the vetoing holders ever rage-quitting. The effective permanent-block threshold for this upgrade therefore drops from 10% toward 1%. This issue was assigned Low severity: it is fail-closed, the worst outcome being a blocked upgrade that can be re-deployed and re-voted (Low impact), and the impact depends entirely on the chosen `EXPIRE_SINCE_INCLUSIVE`, currently a placeholder that can be set with sufficient margin (Low likelihood).

Recommendation

We recommend adding a check at vote creation and/or template deployment that the interval between the current timestamp and `EXPIRE_SINCE_INCLUSIVE` is at least a constant `MIN_EXPIRE_SINCE_INCLUSIVE`, chosen to comfortably exceed the maximum Dual Governance dynamic timelock (currently 49 days: `veto_signalling_max_duration` + `veto_signalling_deactivation_max_duration` + `after_schedule_delay`, plus a safety buffer). This ensures only a genuine RageQuit, which blocks the proposal regardless, can prevent timely execution.

Client's Commentary:

Status: Acknowledged

We intentionally do not want to set a significantly longer expiration period. If the upgrade is delayed or blocked for an extended period of time, we would prefer to prepare and schedule a new governance vote.

The expiration timestamp has been updated to `1786147199` (August 7, 2026, 23:59:59 UTC), providing approximately three weeks from the planned voting start, which we consider sufficient for the expected governance process.

Commit: `9d6a14145265dea095faf4f550c8a84fa445f536`

3. About MixBytes

MixBytes is a blockchain security firm specializing in the analysis of decentralized protocols and smart contract systems.

The company helps Web3 teams build resilient protocol architecture, smart contract logic, and economic mechanisms through a combination of AI-assisted analysis and senior human expertise.

Rather than focusing solely on one-time audits before deployment, MixBytes works with protocols across their entire lifecycle – from early architecture design to production audits and ongoing protocol evolution.

Our team consists of experienced security researchers, engineers, and protocol analysts with deep expertise in DeFi systems, adversarial protocol analysis, and smart contract security.

Over the years, MixBytes has worked with many of the most widely used protocols in the ecosystem, including **Lido**, **Aave**, **Curve**, **1inch**, **OKX**, **Mantle**, **Fluid**, **Gearbox**, **Resolv**, and others, helping teams identify vulnerabilities, strengthen protocol design, and improve the robustness of decentralized systems.

To enhance analysis efficiency and coverage, MixBytes also develops and uses internal AI-assisted tooling that helps surface potential risk signals during development and code review. These tools augment the work of senior auditors but do not replace expert analysis.

Protocol Security Lifecycle

MixBytes supports protocols across the full lifecycle of their development and operation.

- **Design Review.** Independent assessment of protocol architecture and economic design before critical decisions are embedded in production code.
- **AI Tooling.** AI-assisted analysis integrated into development workflows to surface potential risk signals during protocol development.
- **Smart Contract Audit.** Comprehensive manual verification of smart contract logic and protocol invariants before production deployment.
- **Security Retainer.** Continuous expert support for protocol upgrades, integrations, governance changes, and evolving attack surfaces after launch.

Contacts



<https://mixbytes.io/>



https://github.com/mixbytes/audits_public



<https://x.com/MixBytes>



hello@mixbytes.io